

HISTÓRIAS DE USUÁRIO — SQUAD 16

Versão 2.0 — Papéis alinhados ao glossário único de roles

Os papéis utilizados nas histórias abaixo seguem o glossário único definido na Documentação Escrita v2: GAME_MASTER (controle total), PLAYER (amador ou profissional — distinção de UX, não de role no banco), OBSERVER (somente leitura). A distinção amador/profissional reflete o nível de experiência do usuário na interface, não permissões diferentes no sistema.

1. Jogador — Perfil Amador (Role: PLAYER)

História 1.1 — Criar loja facilmente

Role: PLAYER

Descrição: Como um jogador com perfil amador, quero criar minha loja de forma simples para iniciar a simulação rapidamente.

■ v1 não previa o bloqueio para usuários sem squad, que é uma regra definida nas histórias do Game Master.

Critérios de Aceitação:

- Dado que o usuário acessa o sistema pela primeira vez e pertence a um squad ativo, quando clicar em 'Criar Loja' deve ser exibido um formulário com nome e capital inicial
- Dado que o usuário preenche nome e capital inicial válidos, quando confirmar, a loja deve ser criada vinculada ao seu squad
- Dado que a loja foi criada, o usuário deve ser redirecionado para o dashboard da loja
- Se o usuário não pertencer a nenhum squad, o sistema deve bloquear a criação e exibir a mensagem: 'Você precisa estar em um squad para criar uma loja'

História 1.2 — Receber sugestões automáticas

Role: PLAYER

Descrição: Como um jogador com perfil amador, quero receber sugestões de preço e volume de vendas para tomar decisões sem conhecimento avançado.

Critérios de Aceitação:

- Dado que o usuário está configurando a rodada (preenchendo RoundConfigItem), o sistema deve exibir sugestões de salePrice e salesVolume por produto
- As sugestões devem ser baseadas no preço padrão do produto (Product.salePrice) e em 80% do estoque disponível (Inventory.quantity)
- Quando o usuário clicar em 'Aplicar sugestão', os campos de salePrice e salesVolume devem ser preenchidos automaticamente
- Se não houver estoque disponível para um produto, o sistema deve indicar visualmente que ele não pode ser incluído na rodada

História 1.3 — Visualizar resultado simples

Role: PLAYER

Descrição: Como um jogador com perfil amador, quero visualizar o resultado da rodada de forma simplificada para entender se houve lucro ou prejuízo.

■ v1 não tratava o caso de receitaBruta = 0 na exibição, que geraria divisão por zero ou exibição incorreta.

Critérios de Aceitação:

- Se a rodada foi encerrada e o FinancialResult foi gerado, o sistema deve exibir: lucro líquido (netProfit) e margem líquida (netMargin)
- Dado que houve lucro (netProfit > 0), o valor deve aparecer destacado em verde
- Dado que houve prejuízo (netProfit < 0), o valor deve aparecer destacado em vermelho
- Dado que netMargin = 0 (receitaBruta = 0), o sistema deve exibir '—' em vez de percentual

História 1.4 — Entender erros e aprender

Role: PLAYER

Descrição: Como um jogador com perfil amador, quero receber feedback sobre minhas decisões para aprender com meus erros.

Critérios de Aceitação:

- Se netProfit < 0, o sistema deve exibir uma explicação contextualizada (ex: 'Seu preço de venda ficou abaixo do custo de compra')
- Se expenses > grossProfit, o sistema deve alertar: 'Suas despesas consumiram todo o lucro bruto'
- Se effectiveSalesVolume < salesVolume para algum item, o sistema deve informar: 'Volume limitado pelo estoque disponível — revise seu Inventory'
- Os feedbacks devem ser exibidos na tela de resultado após o encerramento da rodada

2. Jogador — Perfil Profissional (Role: PLAYER)

História 2.1 — Definir estratégia completa

Role: PLAYER

Descrição: Como um jogador com perfil profissional, quero configurar preço, volume, estoque e despesas para maximizar meu lucro.

Critérios de Aceitação:

- Na tela de configuração da rodada deve ser possível editar, por produto: salePrice e salesVolume (em RoundConfigItem)
- Na mesma tela deve ser possível editar fixedExpenses e variableExpenses (em RoundConfig)
- O sistema deve exibir o estoque disponível (Inventory.quantity) ao lado do campo salesVolume como referência
- Se salesVolume > Inventory.quantity, o sistema deve exibir aviso em tempo real: 'Volume excede estoque — será limitado automaticamente na simulação'
- Se salePrice < product.purchasePrice, o sistema deve exibir aviso: 'Preço de venda abaixo do custo de compra'
- Se os dados forem inválidos (valores negativos, campos vazios), o sistema deve bloquear o envio com mensagem clara

História 2.2 — Visualizar DRE completo

Role: PLAYER

Descrição: Como um jogador com perfil profissional, quero visualizar um DRE detalhado para analisar minha performance.

■ v1 não especificava a origem do DRE. Recalcular no frontend geraria inconsistência com o que foi persistido no banco.

Critérios de Aceitação:

- O sistema deve exibir o DRE com: Receita Bruta, Custos, Lucro Bruto, Despesas (fixas + variáveis), Lucro Líquido e Margem Líquida
- Cada linha do DRE deve exibir o valor em R\$ formatado

- A Margem Líquida deve ser exibida em percentual (%) com 2 casas decimais
- Se `netMargin = 0` por `receitaBruta = 0`, exibir '—' em vez de '0,00%'
- O DRE deve ser gerado a partir de `FinancialResult` já persistido — não recalculado no frontend

História 2.3 — Simular cenários sem salvar

Role: PLAYER

Descrição: Como um jogador com perfil profissional, quero simular estratégias antes de confirmar para reduzir riscos.

Critérios de Aceitação:

- Quando clicar em 'Simular', o sistema deve chamar `POST /simulation/preview` com os dados atuais do formulário
- O endpoint de preview executa `calcularDRE()` com os dados fornecidos mas NÃO persiste `RoundConfig` nem `FinancialResult`
- O resultado do preview deve ser exibido em uma área separada da tela, claramente marcada como 'Simulação — não confirmada'
- Os dados do preview não devem impactar o ranking nem gerar `FinancialResult` no banco
- O usuário deve poder confirmar a estratégia a partir do preview, que então persiste os dados via `POST /rounds/:id/config`

História 2.4 — Analisar concorrência

Role: PLAYER

Descrição: Como um jogador com perfil profissional, quero visualizar dados dos concorrentes para adaptar minha estratégia.

■ v1 propunha `Squad.score` estático. Corrigido: ranking via `FinancialResult` on-demand.

✓ CORREÇÃO: Ranking servido pelo `GET /ranking?roundId=X`, calculado no `rankingService.js`.

Critérios de Aceitação:

- O sistema deve exibir o ranking da rodada atual com: posição, nome do squad, margem líquida (`netMargin`) e lucro líquido (`netProfit`)
- O ranking deve ser calculado on-demand a partir de `FinancialResult` — não de `Squad.score`
- Os dados individuais de concorrentes (breakdown do DRE) NÃO devem ser expostos — apenas os dados agregados do ranking
- O ranking deve ser atualizado automaticamente após o encerramento de cada rodada

3. Game Master (Role: GAME_MASTER)

História 3.1 — Monitorar squads em tempo real

Role: GAME_MASTER

Descrição: Como Game Master, quero visualizar o desempenho de todos os squads para análise geral da simulação.

Critérios de Aceitação:

- O sistema deve exibir painel com todos os squads: nome, lucro líquido, margem líquida e posição no ranking
- Os dados devem refletir o `FinancialResult` da rodada em andamento ou encerrada mais recente
- O GM deve conseguir filtrar por rodada

História 3.2 — Alterar parâmetros globais

Role: GAME_MASTER

Descrição: Como Game Master, quero ajustar variáveis globais do jogo para influenciar a simulação.

Critérios de Aceitação:

- Dado que o GM altera o purchasePrice de um produto, o novo valor deve ser aplicado a partir da próxima rodada
- Alterações de parâmetros não devem afetar RoundConfig já submetidas na rodada corrente
- O sistema deve registrar log de alterações com timestamp e usuário responsável

História 3.3 — Criar eventos aleatórios

Role: GAME_MASTER

Descrição: Como Game Master, quero inserir eventos aleatórios para tornar o jogo mais dinâmico.

■ A entidade GameEvent não existe na modelagem v1. Precisa ser adicionada à modelagem do banco como trabalho futuro.

Critérios de Aceitação:

- Dado que um evento é criado (ex: 'Aumento de custo de fornecedor'), ele deve ser aplicado como modificador no cálculo da próxima rodada
- O evento deve impactar o motor financeiro: pode alterar purchasePrice, fixedExpenses ou criar um multiplicador em salesVolume
- O evento deve ser visível para os players antes do início da rodada afetada
- Eventos devem ser registrados em tabela própria (GameEvent) — não hardcoded no código

História 3.4 — Gerenciar squads e participantes

Role: GAME_MASTER

Descrição: Como Game Master, quero gerenciar os squads participantes para organizar quem participa da simulação.

■ v1 não previa a validação de líder no DELETE. Fluxo corrigido.

✓ CORREÇÃO: Middleware de DELETE /users/:id deve verificar User.leader antes de prosseguir.

Critérios de Aceitação:

- O GM deve conseguir criar, editar e remover squads
- O GM deve conseguir adicionar ou remover usuários de um squad
- O sistema deve impedir usuários com role PLAYER sem squadId de submeter RoundConfig
- Ao remover um usuário que é leader de um squad (User.leader = true), o sistema deve exigir a transferência de liderança antes de prosseguir com a remoção
- Alterações de squad devem ser refletidas antes do início de uma nova rodada — não durante rodada com status OPEN ou PROCESSING